OBJECT ORIENTED PROGRAMMING

Assignment 2

Topic: Class Handling in C++ Duration: 1.5 Week

Total Marks: 100 Submission Deadline: 25-11-2025

CLO	CLO Statement	Blooms Taxonomy Level	PLO

Task 1:

Scenario: Student Registry (Class + File Persistence)

Design a menu-driven Student Registry system that persists data to disk. Use a Student class with proper encapsulation and implement CSV and binary read/write operations using fstream.

Menu Options

1. Add Student
2. Display All
3. Save to CSV
4. Load from CSV
5. Save to Binary
6. Load from Binary
7. Search (by ID / by Partial Name)
8. Update (by ID)
9. Delete (by ID)
10. Exit

Tasks and Marks

Task 1: Class & Validation (15 Marks)

Implement constructors and validation:
- id > 0, non-empty name, age ≥ 16, gpa in [0,4].
- Reject invalid input and prompt again.

Demonstrate by creating objects and printing them.

Task 2: CSV I/O (25 Marks)

Implement toCSV() and fromCSV() with parsing and trimming.
Save and load data in CSV format, skipping invalid lines.
Show replace/merge behavior and load summary.

Task 3: Binary I/O (20 Marks)
Use writeBinary()/readBinary() for safe serialization.
Handle stream errors and EOF gracefully when reading.

Task 4: Search, Update, Delete (20 Marks)
Implement:
- Search by ID and by name (case-insensitive).
- Update name, age, or GPA (validate).
- Delete by ID, compact array or shift left.

Task 5: Robustness & UX (10 Marks)
Ensure clean input handling, table formatting, and confirmation before delete/overwrite.
Show counts and summaries after every operation.

Task 6: Report & Demo (10 Marks)
Add header comments explaining CSV/Binary formats and validation rules.
Provide sample CSV and BIN files.

Task 2:

Scenario: Banking System Simulation
Design a small Banking System with a BankAccount class and additional helper classes. Each section builds on the previous one.

Question 1 — Constructors, Destructors, and Class Interface (15 Marks)
Create a class BankAccount that contains:
Data Members:
- int accountNumber
- std::string holderName
- float balance

Tasks:
1. Write a default constructor, parameterized constructor, and a destructor.
2. Create member functions:
   - void deposit(float amount)
   - void withdraw(float amount)
   - void display() const
3. Demonstrate creating objects and destructor call on scope exit.

Question 2 — Passing and Returning Objects, Copy Constructor, Deep Copy (20 Marks)

Enhance the BankAccount class:
1. Add a copy constructor that performs a deep copy.
2. Write a function BankAccount mergeAccounts(const BankAccount& a1, const BankAccount& a2).
3. Show passing objects by value and reference.
4. Demonstrate shallow vs deep copy through examples.

Question 3 — The this Pointer, Constant & Static Members (20 Marks)

Add new features to BankAccount:
1. Add static members: totalAccounts, totalBankBalance.
2. Update constructors/destructor to modify static variables.
3. Add const function showSummary() const.
4. Add static function showBankStats().
5. Use this pointer in deposit() and withdraw().
6. Demonstrate const object usage and static function call.

Question 4 — Arrays, Pointers to Objects, and Dynamic Allocation (25 Marks)

Extend your system to handle multiple accounts.
1. Create an array of BankAccount objects.
2. Use pointer to object and dynamic arrays.
3. Input/display all using pointer arithmetic.
4. Demonstrate array of pointers to objects and delete all dynamically allocated objects.

Question 5 — Friend Functions and Friend Classes (20 Marks)

Create a class Loan with:
- int loanId
- float amount
- float interestRate
- int durationMonths

Tasks:

1. Make a friend function showLoanDetails(const BankAccount&, const Loan&).
2. Make Loan a friend class of BankAccount.
3. Demonstrate creating Loan object and using friend access.

Deliverables

- A single project file: main.cpp (or separate headers if preferred).
- Must compile and run cleanly.
- Output must clearly show each concept in action.
- Include comments explaining:
  - Constructor/Destructor sequence
  - Deep vs Shallow copy difference
  - Static member behavior
  - Friend function access

Evaluation Rubric

Section	Marks	Key Skills Tested
Q1: Constructors & Interface	15	Class syntax, constructor/destructor, scope
Q2: Copy Mechanics	20	Passing/returning objects, deep copy
Q3: this, const, static	20	Static/const usage, this pointer
Q4: Arrays & Pointers to Objects	25	Memory management, dynamic access
Q5: Friend Functions & Classes	20	Controlled access between classes